FP Exercise Sheet
Week 4: Evaluation Strategies and Polymorphism

October 13, 2025

Parametric Polymorphism

1. Write a function `rotate :: (a, b, c) -> (c, a, b)`.
2. Write a function `myCurry :: ((a, b) -> c) -> a -> b -> c`.
3. Write a function `myUncurry :: (a -> b -> c) -> (a, b) -> c`.
4. Write a function `myFlip :: (a -> b -> c) -> b -> a -> c`.
5. Write a function `f :: (a, b) -> (a -> c) -> (b -> d) -> ((c, d) -> e) -> e`.

Typeclasses

1. Define a `Vehicle` ADT with values that are various kinds of vehicles, perhaps `Bicycle`, `Car` and `Bus`.
2. How will you modify the ADT definition so you can 'see' `Vehicle` values in GHCi?
3. How will you modify the ADT definition so you can compare `Vehicle` values for equality with ==?
4. Now define a function `numWheels :: Vehicle -> Int` for Vehicle objects, with appropriate clauses.
5. The `Ord` typeclass is used for values that can be compared with comparison operators (really functions) like <= or >. Types that are instances of `Ord` must provide an implementation of the `compare :: Ord a=>a -> a -> Ordering` function. Check out the definition of `Ordering` then make `Vehicle` an instance of `Ord` using the number of wheels as the ordering metric, so `Car > Bicycle` for instance.

Lazy Evaluation

We looked at the recursive `fib` function as an example of an expensive expression to evaluate. Other recursive functions are even more expensive, e.g. the *Ackermann* function, shown below:

```
-- Ackermann function
-- its value grows very rapidly
-- and it takes a _long_ time to compute
-- when the first argument is >= 4
ackermann :: Integer -> Integer -> Integer
ackermann 0 n = n + 1
ackermann m 0 = ackermann (m - 1) 1
ackermann m n = ackermann (m - 1) (ackermann m (n - 1))
```

1. Define a `butIfZero :: Integer -> Integer -> Integer` function. This function should return its first parameter so long as this is non-zero, otherwise the function returns its second parameter. So for instance:

```
(1+1) `butIfZero` (ackermann 4 2)
```

   should evaluate rapidly to 2, but on the other hand...

```
(ackermann 4 2) `butIfZero` (1+1)
```

   should take a long time to evaluate. (Press Ctrl+C to quit the evaluation.) Can you explain the **reason** for this difference? How would you **describe** function `butIfZero` in terms of the evaluation of its parameters?

2. Now we are going to experiment with *strictness annotations*. Consider the following Haskell function definition:

```
let f = \x y z -> if (x=="hello" && y=="haskell")
                  then z
                  else 42
```

   (a) What is the type of `f`?

   (b) How would you describe function `f` in terms of the evaluation of its parameters? Which parameters (if any) must *always* be evaluated?

   (c) Would you expect the following function evaluation to take a long time? Why (or why not)?

```
f "hello" "c++" (ackermann 4 2)
```

   (d) Adding a bang character '!' before a parameter name forces evaluation of this parameter at the function call site. '!' is called a *strictness annotation*. So now look at this modified definition:

```
let f = \x y !z -> if (x=="hello" && y=="haskell")
                   then z
                   else 42
```

   Would you expect this function evaluation to take a long time? Why (or why not)?

```
f "goodbye" "java" (ackermann 4 2)
```

   (e) Can you see any useful reason for strictness annotations in Haskell?

Further Exercises

Random numbers can be generated using a *pseudo-random number generator*[1] (PRNG). A PRNG is created using a *seed* value. When we want to generate a random number, we provide the current PRNG and get back a pair of a pseudo-random value and a new generator.

The only impure part of random number generation is the creation of the PRNG with a random-ish seed (we could use, e.g., the current system timestamp or random mouse movements). For now, let's sidestep the impurity issue by using a deterministic seed.

From the command line, install the `random` library by writing `cabal install random`. Take a look at the documentation for `System.Random`[2]. Useful functions include:

- `mkStdGen :: Int -> StdGen`, which constructs a PRNG from a given seed.
- `uniformR :: (RandomGen g, UniformRange a) =>(a, a) -> g -> (a, g)`, which generates a random number in the range given by the tuple along with a new random generator. (Note that `StdGen` is a member of the `RandomGen` typeclass).

[1] see `https://en.wikipedia.org/wiki/Pseudorandom_number_generator`

[2] `http://hackage-content.haskell.org/package/random-1.3.1/docs/System-Random.html`

1. Write a function `randomElem :: (RandomGen g) =>g -> [a] -> (a, g)` that uses a random number generator `g` to return a random item of a list. You can assume that the input list is non-empty.

2. Write a function `randomElements :: Int -> [a] -> [a]` that takes a seed, a non-empty source list, and produces an infinite list of randomly-selected elements from the source list.

   As an example:

   ```
   ghci> take 20 (randomElements 135435 [1..20])
   [8,11,7,13,1,9,9,20,9,6,4,8,7,6,17,8,11,5,11,17]
   ```